

LINUX / OPERATING SYSTEM LAB ASSIGNMENT

1. What If There Were No Operating System?

Imagine a computer without an operating system. The hardware may be present, but using it would be very difficult because there would be no software layer to manage the hardware and provide an interface for the user.

Without OS	With OS
User cannot easily interact with hardware	OS provides CLI/GUI to interact with the computer
Every program would need to control hardware directly	OS manages hardware resources
Files would be difficult to create, organize and access	OS provides a file system
Programs could conflict while using RAM/CPU	OS manages memory and CPU
No convenient way to run multiple applications	OS manages processes and multitasking
Devices such as keyboards, disks and printers would be difficult to use	OS provides device drivers and device management

Three OS Features Demonstrated Using Linux

Feature 1 - File Management

Linux organizes files using directories. The following commands demonstrate basic file management:

```
mkdir os_lab
```

```
cd os_lab
```

```
touch test.txt
```

```
ls
```

Observation: mkdir creates a directory, cd moves into it, touch creates a file and ls displays the contents.

Without OS: The user would have to directly communicate with storage hardware to create and locate files.

With OS: Linux provides a simple file-system interface for managing files.

Feature 2 – Process Management

Linux allows users to view and manage running processes.

```
ps
```

```
ps aux
```

Observation: These commands display processes currently running on the system.

Without OS: Programs would have to manage CPU resources themselves, which could cause conflicts.

With OS: Linux schedules processes and allocates CPU resources between them.

Feature 3 – Device / Hardware Management

Linux provides commands to inspect hardware devices.

lsblk

lspci

Observation: lsblk displays available block storage devices, while lspci displays PCI devices connected to the computer.

Without OS: Hardware would be difficult for normal users and applications to access directly.

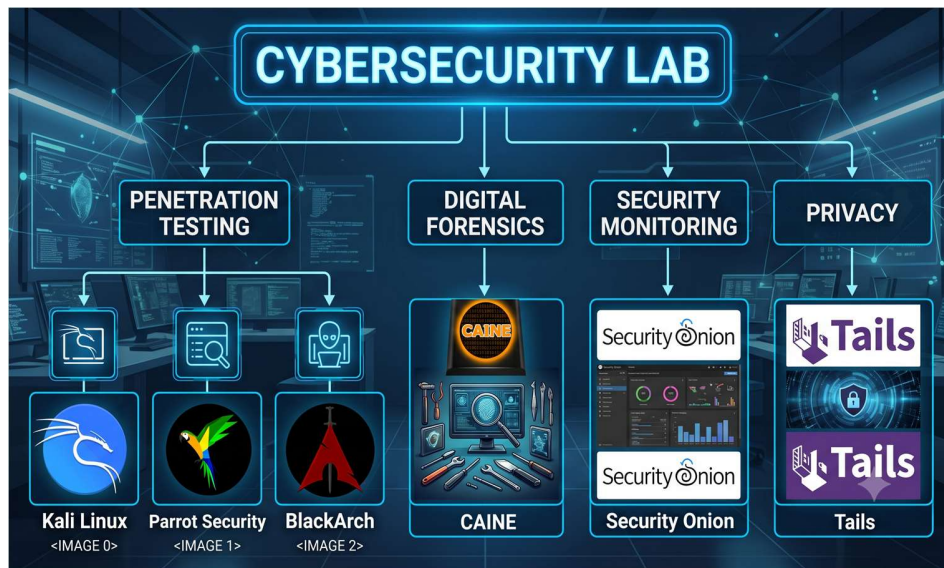
With OS: Linux provides standard interfaces and tools for accessing and inspecting hardware.

2. Linux Distributions for a Cybersecurity Laboratory

Different Linux distributions are designed for different cybersecurity activities. Some already contain penetration-testing tools, while others are better suited to digital forensics, privacy or security monitoring.

Linux Distribution	Main Purpose	Important Features	Suitable Role
Kali Linux	Penetration testing	Nmap, Metasploit, Wireshark, Burp Suite and many security tools	Penetration Tester
Parrot Security OS	Security testing & privacy	Security tools, lightweight environment, privacy features	Penetration Testing / Security Research
BlackArch Linux	Advanced penetration testing	Large collection of security tools	Advanced Security Research
Tails	Privacy and anonymity	Runs from removable media; privacy-focused design	Privacy & Anonymous Research
Security Onion	Security monitoring	Network monitoring, intrusion detection and analysis tools	SOC / Security Monitoring
CAINE	Digital forensics	Forensic investigation and evidence-analysis tools	Digital Forensics

OS Selection Guide



Recommended Choices

Penetration testing → Kali Linux

Kali would be my first choice because it is specifically designed for penetration testing and security auditing and provides a large collection of security tools.

Digital forensics → CAINE

CAINE is suitable for forensic investigations because it provides tools for examining digital evidence.

Privacy → Tails

Tails is designed around privacy and anonymity and can be run from removable media.

Security monitoring → Security Onion

Security Onion is suitable for building a security monitoring/SOC environment because it focuses on network security monitoring, detection and analysis.

Advanced security research → BlackArch

BlackArch provides a very large collection of security tools and is more suitable for experienced Linux and cybersecurity users.

My Cybersecurity Lab Selection

- Kali Linux → Penetration testing
- CAINE → Digital forensics
- Tails → Privacy testing

- Security Onion → Security monitoring
- Parrot Security OS → General security research

This combination covers several important areas of cybersecurity instead of depending on only one distribution.

3. Linux Exit Status

An exit status is a value returned by a Linux command after it finishes executing. The special variable \$? contains the exit status of the most recently executed command.

Generally:

- 0 → Command was successful
- Non-zero value → Command failed or encountered some condition

Test 1 – pwd

```
pwd
```

```
echo $?
```

Example result: pwd

Exit status: 0

Meaning: Command successfully displayed the current directory.

Test 2 – ls

```
ls
```

```
echo $?
```

Example result: ls

Exit status: 0

Meaning: Directory contents were successfully displayed.

Test 3 – ls nonexistentfile

```
ls nonexistentfile
```

```
echo $?
```

Example result: ls: cannot access 'nonexistentfile': No such file or directory

Exit status: 2

Meaning: The specified file does not exist.

Test 4 – mkdir testfolder

mkdir testfolder

echo \$?

Example result: Directory created

Exit status: 0

Meaning: The directory was successfully created.

Test 5 – cd wrongfolder

cd wrongfolder

echo \$?

Example result: bash: cd: wrongfolder: No such file or directory

Exit status: 1

Meaning: The directory could not be accessed.

Command	Result	Exit Status	Meaning
pwd	Successful	0	Current directory displayed
ls	Successful	0	Directory contents displayed
ls nonexistentfile	Failed	2	File does not exist
mkdir testfolder	Successful	0	Directory created
cd wrongfolder	Failed	1	Directory could not be accessed

What Can a Script Learn from an Exit Status?

A shell script can use an exit status to determine whether a command worked or failed. For example:

```
mkdir backup
```

```
if [ $? -eq 0 ]
```

```
then
```

```
    echo "Backup directory created successfully"
```

```
else
```

```
    echo "Failed to create backup directory"
```

```
fi
```

A more common approach is:

```
if mkdir backup
```

```
then
```

```
    echo "Directory created successfully"
```

```
else
```

```
    echo "Directory creation failed"
```

```
fi
```